AskTheVideo

Ask any question about a YouTube video

Krzysztof Giwojno

IronHack AI Engineering Bootcamp 2026

The problem

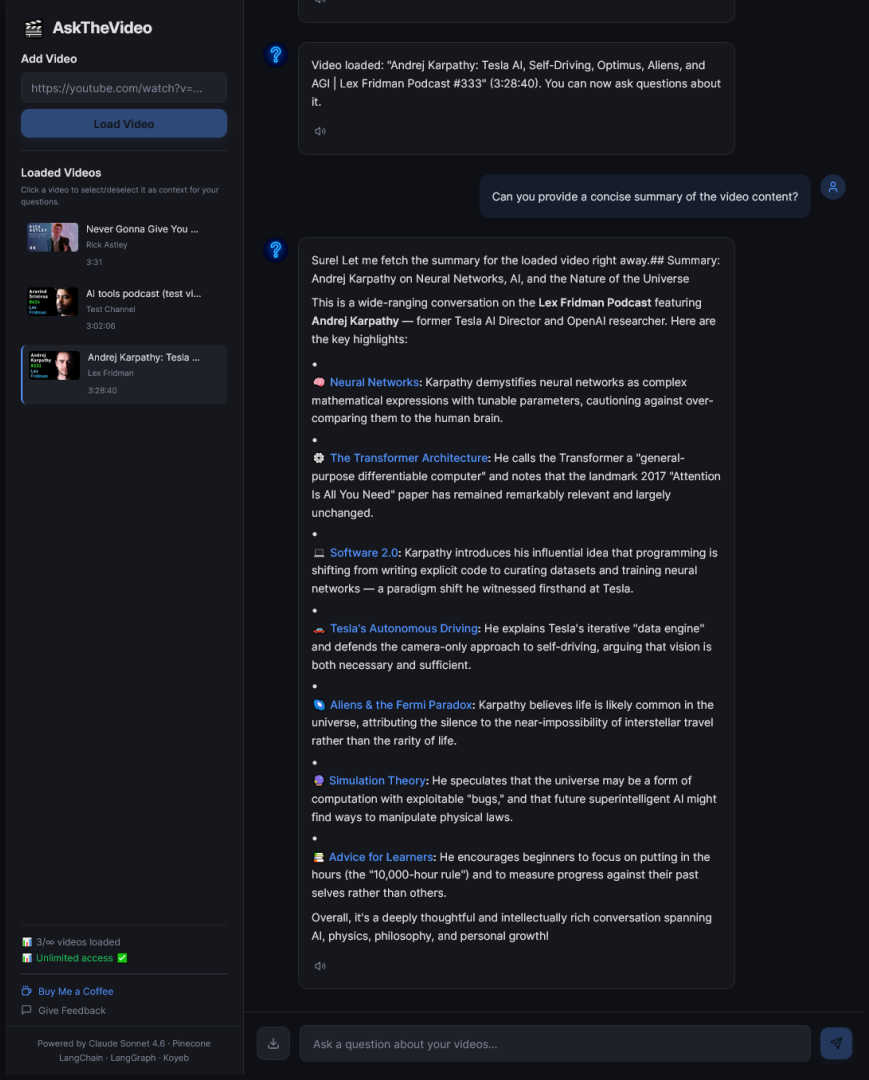
"I watched a 3-hour podcast but I can't find that one thing they said."

"There are 50 videos on this topic. I don't have time to watch them all."

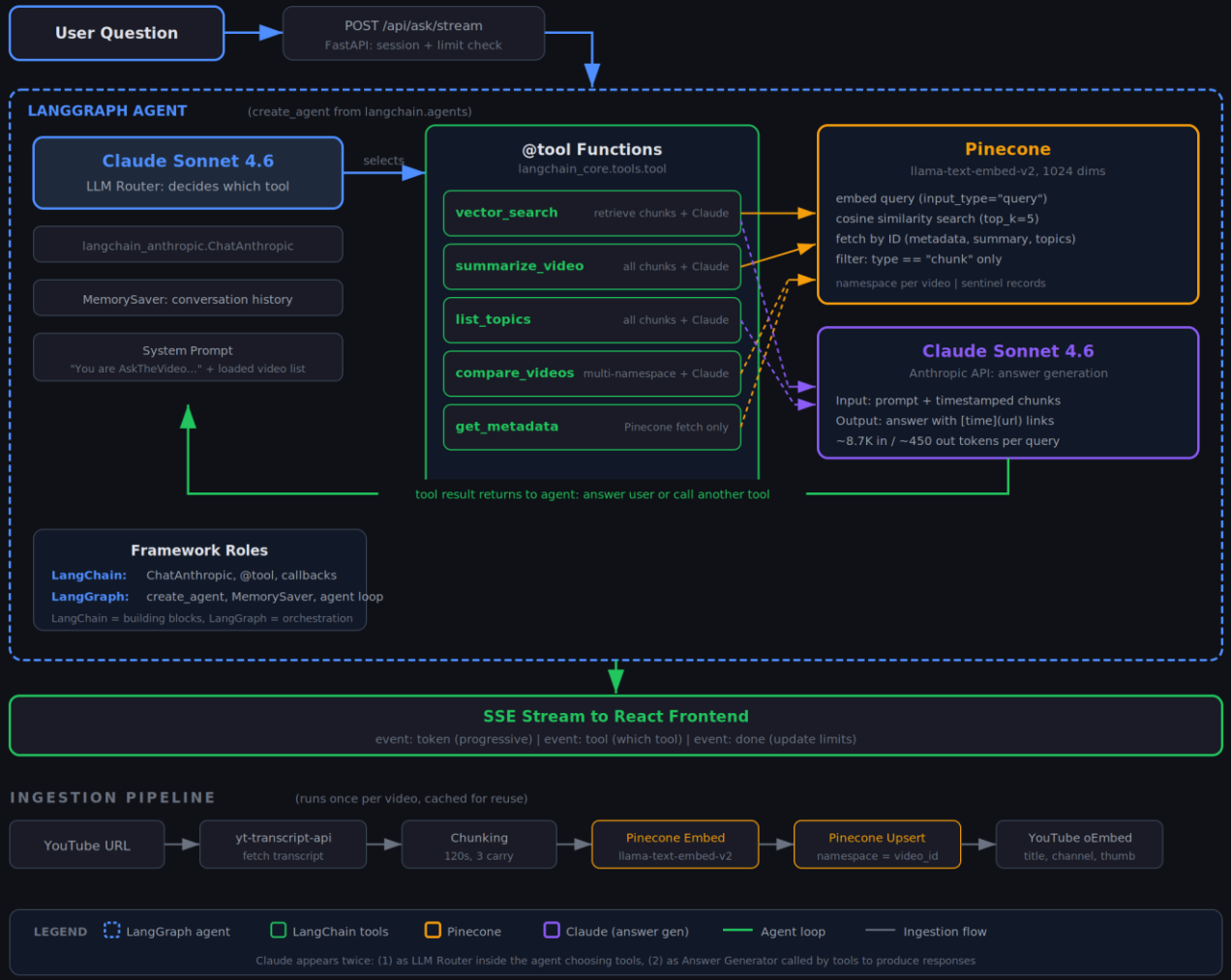
The solution

Paste a YouTube URL. Ask a question.
Get an answer with the exact timestamp.

`Live at askthevideo.com`



AskTheVideo — AI Agent Flow



Dataset / Ingestion Pipeline

120s chunk window with 3-snippet carry-over

1,024 embedding dimensions (llama-text-embed-v2)

0.4s cache hit (vs 2-10s fresh ingestion)

$0.00 embedding cost (Pinecone free tier)

Source

youtube-transcript-api via Webshare residential proxy

Chunking

120-second sliding window
3 transcript snippets carried over for context continuity
Started at 60-90s, increased after testing retrieval quality
Result: better context per chunk, timestamps still precise enough

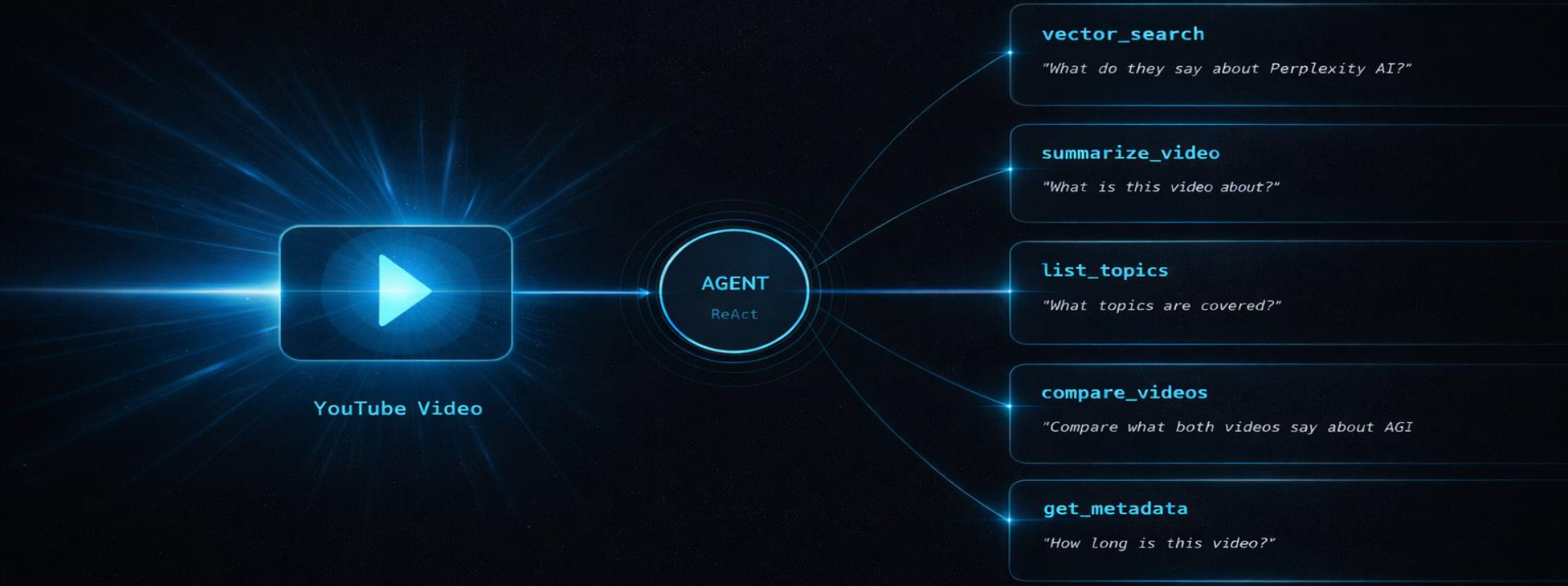
Storage

Pinecone serverless, one namespace per video
Sentinel-vector records for cached summaries and topics

Cross-user reuse

If anyone has loaded a video before,
it loads instantly for the next user

One video. Five tools. The right answer every time.



The tool descriptions are the most important piece of code. Precise descriptions: 100% routing accuracy.

LangGraph ReAct Agent | Claude Sonnet 4.6 | MemorySaver | SSE streaming

Five tools

`vector_search`	Find relevant excerpts, answer with timestamps	Claude: yes	Cache: no
`summarize_video`	Full summary with key points and timestamps	Claude: yes	Cache: yes
`list_topics`	Extract 8-12 topics with timestamp ranges	Claude: yes	Cache: yes
`compare_videos`	Compare what multiple videos say about a topic	Claude: yes	Cache: no
`get_metadata`	Video title, channel, duration, thumbnail	Claude: no	Cache: n/a

Caching impact (summaries and topics)

First call:	~22 seconds	~$0.14
Cached call:	**~0.6 seconds**	**$0.00**

What I shipped

SSE Streaming

Token-by-token real-time responses

Voice Input

Ask questions by speaking

Voice Output

Listen to answers read aloud

Chat Export

Download conversation as markdown

Responsive Design

Desktop, tablet, and mobile

Session Persistence

Refresh the page, session survives

Conversation Memory

Follow-up questions work naturally

User Analytics

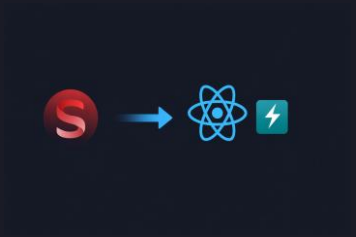
Anonymous tracking, returning users

Admin Dashboard

Live metrics, cost, events, filters

None of these were in the original plan. More detail in appendix.

Challenges



Streamlit could not match the design.

No SSE streaming. No voice APIs.

Poor mobile experience.

Pivoted to React + FastAPI.

All backend code survived untouched.

3 proxy attempts failed (half a day lost).

All datacenter IPs blocked.

Residential proxy. 10 lines of code.

More detail on both challenges in appendix.

Evaluation

Routing accuracy	14/16 (88%)
Answer quality	2.8/3.0 avg
Hallucinations	0/16
Timestamp accuracy	12/14 (86%)
Keyword recall	74%
Retrieval (top-1)	0.404 cosine

16 questions, 2 videos (5.5 hours total)
Ground truth from manual video review

Zero hallucinations.

When the agent cannot find relevant information, it says so rather than inventing an answer.

One routing "miss" was the agent interpreting the question better than my test expected.

Routing accuracy:

88% or 94%,
depending on how you count.
Either way: zero wrong answers.

Full evaluation breakdown and methodology in appendix.

What it cost to build

Actual spend

Claude.ai + Claude Code	~$40
Anthropic API (dev + eval)	~$8
Domain + hosting (1 year)	~$10
Webshare proxy (1 month)	$3.50
Total	**~$62**

Per-session cost

~$0.25-0.30 for 10 questions

Most bootcamp projects prove you can build something. I wanted to ship a product. Deployed, monitored, and ready for real users. The spend reflects that ambition.

Tech stack decisions and cost rationale in appendix.

Free (would have cost)

Lovable (React frontend)
Free build day. Would be ~€50/month

Genuinely free tier

Pinecone (DB + embeddings)

Koyeb (Docker hosting)

LangSmith (tracing)

Google Analytics

GitHub (3 repos)

What I built

3 repositories (backend, frontend, landing page)

~1,750 lines of production Python

7 Jupyter notebooks

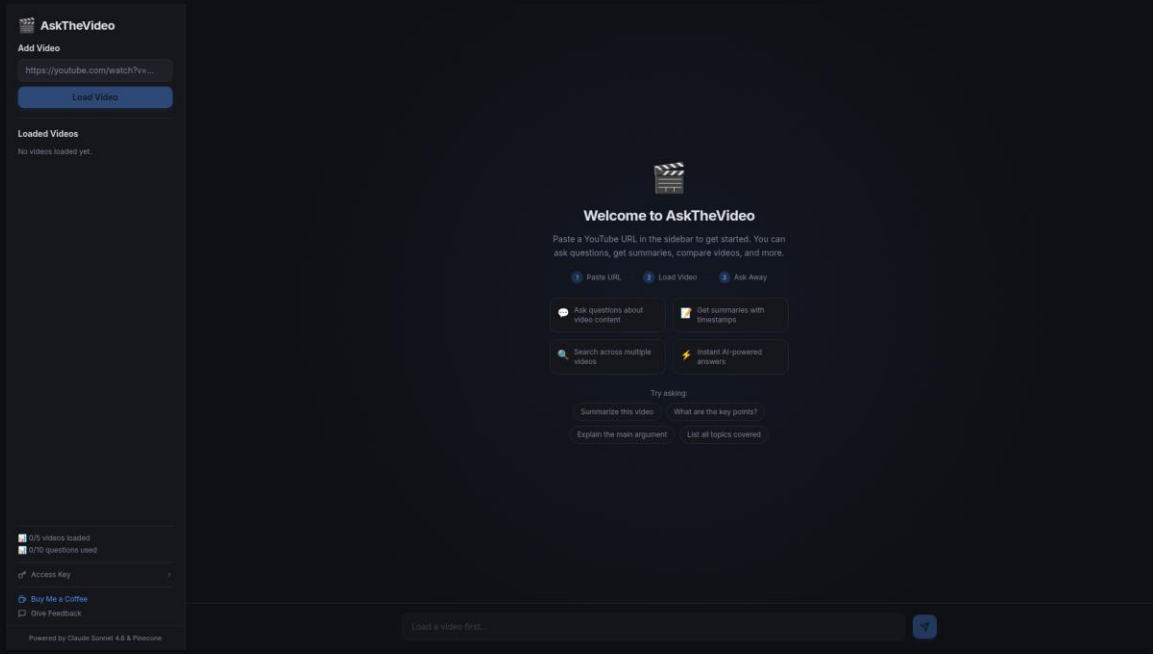
11 API endpoints, 5 AI tools

51 unit tests + 15 smoke tests

30 documented deviations from spec

Admin dashboard with live monitoring

Built solo in ~10 days



`#2 on Google for "ask the video"`

`askthevideo.com`

Thank you.

Demo

app.askthevideo.com

AskTheVideo

Add Video

https://youtube.com/watch?v=...

Load Video

Loaded Videos

No videos loaded yet.

Unlimited access ✅

Buy Me a Coffee

Give Feedback

Powered by Claude Sonnet 4.6 · Pinecone
LangChain · LangGraph · Koyeb

Welcome to AskTheVideo

Paste a YouTube URL in the sidebar to get started. You can ask questions, get summaries, compare videos, and more.

1 Paste URL → 2 Load Video → 3 Ask Away

- Ask questions about video content
- Get summaries with timestamps
- Search across multiple videos
- Instant AI-powered answers

Try asking:

- Summarise this video
- List all topics covered
- What are the most important moments?
- Compare what the selected videos say about topic of

Load a video first...

https://drive.google.com/file/d/1YedbhMjnMx19KkzSCXuUkXBlVTeZpFyM/view?usp=drive_link

askthevideo.com

Questions?

Evaluation Results at a Glance

Metric	Result	Notes
Routing accuracy	14/16 (88%)	2 misses are ambiguity, not errors
Answer quality	2.8/3.0 avg	12 correct, 4 partial, 0 wrong
Keyword recall	74%	Agent paraphrases rather than echoing transcript
Timestamp accuracy	12/14 (86%)	Verified against raw transcript text
Hallucinations	0/16	No fabricated content in any answer
Retrieval top-1	0.404 cosine	On threshold, strong for spoken content
Retrieval consistency	0.058 drop-off	Top results tightly clustered, not lucky hits
Relevant in top-3	7/8	1 miss on broad query pattern

Zero hallucinations is the most important finding. The RAG architecture with forced citations works as designed. When the agent cannot find relevant information, it says so explicitly rather than inventing an answer.

Per-Question Breakdown (Q1-Q16)

ID	Tool used	Question	Route	Keywords	Timestamps
Q1	vector_search	Tesla's Autopilot data pipeline?	OK	75%	4
Q2	vector_search	Karpathy's AGI timelines?	OK	67%	2
Q3	vector_search	Neural network training?	OK	100%	5
Q4	compare*	Perplexity vs Google?	MISS	75%	13
Q5	vector_search	Origin of Perplexity?	OK	33%	2
Q6	vector_search	Google's ad business model?	OK	100%	5
Q7	vector_search	History of transformers?	OK	100%	8
Q8	vector_search	Startup advice?	OK	100%	7
Q9	summarize	Summarise Karpathy video	OK	60%	7
Q10	summarize	Perplexity video about?	OK	100%	7
Q11	list_topics	Topics covered? (Karpathy)	OK	-	-
Q12	list_topics	Outline of video?	OK	-	-
Q13	compare	Both guests on AGI?	OK	-	-
Q14	compare	Future of AI?	OK	-	-
Q15	compare*	What videos loaded?	MISS	-	-
Q16	get_metadata	How long is Karpathy?	OK	-	-

** = routed to wrong tool but produced correct answer. Q4: compare_videos found 13 timestamps across both videos. Q15: compare_videos listed both video IDs correctly.*

Retrieval Quality Analysis

Per-question cosine scores (top-1)

	Score	Relevant in top-3
Q1: Tesla Autopilot data	0.339	1/3
Q2: AGI timelines	0.400	2/3
Q3: Neural network training	0.349	2/3
Q4: Perplexity vs Google	0.552	2/3
Q5: Origin of Perplexity	0.448	1/3
Q6: Google ad business	0.516	2/3
Q7: History of transformers	0.219	3/3
Q8: Startup advice	0.408	0/3

Summary Statistics

Avg top-1 cosine	0.404	target: > 0.4
Avg top-3 cosine	0.383	target: > 0.35
Avg score drop-off	0.058	target: < 0.15
Relevant in top-3	7/8	target: all

Top-k Comparison (5 vs 10)

All three tested questions show small gaps between top-5 and extra 6-10 chunks. Chunks 6-10 add useful context, not noise. Current top_k=5 is a good cost/quality balance.

Q1: gap 0.022 Q4: gap 0.050 Q7: gap 0.016

Q7 (0.219) is misleading: all three top chunks were from the correct section. Abstract queries score low against specific text despite correct retrieval. V3 (auto-captions) averaged lower than V4 (manual captions with punctuation).

Evaluation Methodology

Setup

16 questions across 2 videos:
V3: Lex Fridman / Karpathy (2h 35m, 104 chunks)
V4: Lex Fridman / Perplexity (3h 02m, 90 chunks)

Question distribution:
8 vector_search, 2 summarize_video,
2 list_topics, 2 compare_videos, 2 get_metadata

Ground truth: manual video review
(watched ~20 min of V3, V4 known from notebooks)

Timestamps verified by clicking every link
and cross-referencing against raw transcript text.

Bug discovered during evaluation:

Rate limit on Q11 corrupted MemorySaver state (tool_use without tool_result). All subsequent questions failed. Fixed post-evaluation: all 5 tool wrappers now catch exceptions.

Evaluation Runs

Run 1 ~$0.80
Q1-Q10 success, Q11 rate limited,
Q12-Q16 cascade failure

Run 2 ~$0.80
Q1-Q10 success, Q11 rate limited,
Q12-Q16 cascade failure

Run 3 ~$0.40
Fresh session, 30s delays.
Q11-Q16 all success.

Retrieval $0.00
8 queries, Pinecone only.
No Claude calls.

Total evaluation cost: $2.20

YouTube Blocks Cloud Servers

The problem: youtube-transcript-api works perfectly on localhost. The moment you deploy to any cloud provider (Koyeb, AWS, GCP, Cloudflare), every transcript request fails with IpBlocked. YouTube blocks all datacenter IP ranges.

Attempt 1: Cloudflare Worker

Built a full Worker (scripts/cloudflare-worker.js) using YouTube's internal Innertube API. Tried Android, WEB, IOS, TVHTML5 clients. The timedtext endpoint returns 0 bytes from Cloudflare IPs. YouTube blocks at multiple levels: player API, timedtext, and get_transcript.

Attempt 2: Google Cloud Function

Built a GCF proxy (scripts/gcf-transcript-proxy/main.py) with the same Innertube approach. Theory: Google owns YouTube, so maybe GCF IPs aren't blocked. Pivoted before deploying - same timedtext endpoint would be blocked.

Attempt 3: yt-dlp Fallback

Added yt-dlp as a fallback. Different code path (player page parsing + subtitle URLs). But yt-dlp ultimately hits the same timedtext endpoint. Datacenter IPs are datacenter IPs.

Solution: Webshare

Rotating residential proxy.
Real ISP IP addresses.
YouTube does not block residential IPs.

GenericProxyConfig with
p.webshare.io:80
-rotate username suffix

10 lines of code.
$3.50/month.
Free tier: 10 rotating IPs.

Lesson: YouTube's IP blocking is comprehensive. No amount of API cleverness (different clients, endpoints, headers) bypasses it from datacenter IPs. The only reliable solution is residential IP addresses. Half a day lost on three approaches. The solution that worked took 10 lines.

Streamlit to React + FastAPI

What stayed (zero changes)

All src/ modules:
transcript.py, chunking.py, vectorstore.py,
tools.py, agent.py, metadata.py, validation.py

Pinecone, Anthropic, LangGraph, LangChain
All 67 decisions from Phase 1
Koyeb deployment target
Session management pattern (ephemeral)

What changed

Streamlit	->	React (Lovable)
st.session_state	->	In-memory dict + UUID headers
Global loaded_videos	->	Per-request closures
Streamlit page	->	React /admin route + API
Port 8501	->	Port 8000
~90MB RAM	->	~35MB RAM

Why the pivot happened

Designed the UI in Lovable (free AI tool that generates React). Looked excellent in Lovable's preview. Applied the same design to Streamlit via CSS overrides and custom HTML injection. Result: significantly degraded. Streamlit controls its own DOM, limits styling, and many CSS overrides produce inconsistent results.

Core problem: Lovable builds beautiful React. Streamlit's styling is fundamentally constrained.

New features enabled by React: SSE streaming, voice input/output, chat export, mobile responsive, client-side routing for admin panel.

Known Limitations

Multi-video ambiguity	Vague questions with 2+ videos loaded cause agent to ask for clarification instead of answering
Auto-caption quality	Videos with manual captions produce higher cosine scores than auto-generated (no punctuation, filler words)
Rate limiting on heavy tools	list_topics and compare_videos send all chunks to Claude, can exceed 30K input tokens/min
Hardcoded model name	src/agent.py hardcodes claude-sonnet-4-6 instead of importing from config/settings.py
Duplicate constants	Same constants in config/settings.py, src/tools.py, and src/vectorstore.py (notebook extraction pattern)
No real frontend tests	Only a dummy test exists. No unit or integration tests for React components.
YouTube ToS risk	youtube-transcript-api scrapes internal endpoint, not official API. Could be blocked.

Future Improvements

- Whisper fallback for videos without captions
- Namespace TTL cleanup using ingested_at metadata
- Auth-based user tracking (beyond anonymous UUIDs)
- Horizontal scaling (paid Koyeb tier)
- Migrate to YouTube Data API v3 (eliminate ToS risk)
- Full GDPR remediation (cookie consent, privacy policy)

LinkedIn Post Series

Tuesday (pre)	Teaser	OG image (cinematic)	Build curiosity, drive traffic to teaser page
Thursday (pre)	Insight	Agent routing flow (cinematic)	Position as thought leader, technical credibility
Friday	Presentation	N/A	Deliver final project to bootcamp
Tuesday (post)	Full reveal	TBD	Full project reveal with demo, link to live app

Strategy

Visual continuity: same cinematic palette (dark navy, cyan glow) across all posts and the landing page. Creates instant recognition.

Two-step approach for cinematic visuals: matplotlib blueprint for layout, then ChatGPT image generation for cinematic rendering.

Posting time: 8-9am Berlin. Catches European morning, late evening Asia, early morning US East Coast.

No reveal before presentation: avoid unexpected traffic and API credit consumption before the live demo.

SEO Results

#2 on Google for "ask the video"

Achieved with just a teaser page:
- Semantic HTML5 + proper meta tags
- Open Graph + Twitter Cards
- Full favicon set + PWA manifest
- JSON-LD structured data
- HTTPS redirect + canonical URL
- Google Search Console verification
- Google Analytics (G-KY52VN1JCR)

Domain: askthevideo.com
App: app.askthevideo.com

Why These Technologies

Claude Sonnet 4.6	Latest model (Feb 17, 2026). Near-Opus quality at Sonnet pricing ($3/$15 per MTok). Better instruction following, fewer hallucinations than 4.5. 200K context.
Pinecone	Free tier covers everything: 5 indexes, 2GB storage, 2M writes/mo, 1M reads/mo. Cloud-hosted, no self-managed infra. Inference API for embeddings (also free).
llama-text-embed-v2 1024 dimensions	Matryoshka Representation Learning: 1024 = 98-99% of full 2048 quality. Zero-config (Pinecone default). ~10,000 videos fit in 2GB at 1024 dims vs ~5,000 at 2048. Spoken transcripts don't need the last 1-2%.
LangGraph + LangChain	LangChain provides LLM wrappers, @tool decorators, Pinecone integration. LangGraph provides agent orchestration, MemorySaver, and the ReAct agent loop. Together: tool routing + conversation memory + multi-step reasoning.
Koyeb	Free Docker hosting. Auto-deploy on push to main. 512MB RAM, 0.1 vCPU. Custom domain support. Frankfurt region. Single container serves both frontend and API.
React (Lovable)	Lovable generates production React for free (build day). Full design control, SSE streaming, Web Speech API, client-side routing. Replaced Streamlit after design constraints became clear.

74 documented decisions in PROJECT_PLAN.md. Every choice has a rationale.

Admin Dashboard

AskTheVideo — Admin Dashboard

Refresh

REAL-TIME

1	105 / 512 MB	0.0%	0.7h
ACTIVE SESSIONS	RAM USAGE	CPU	UPTIME

SESSIONS

1	6	1	0 / 0
TOTAL QUERIES	VIDEOS LOADED	KEY QUERIES	ERRORS / ALERTS

USERS

1	0	1.0	1.0
TOTAL USERS	RETURNING USERS	AVG SESSIONS/USER	AVG QUESTIONS/USER

API COST

1,788,909	152,988	$7.67	Cycle Budget
INPUT TOKENS	OUTPUT TOKENS	ESTIMATED COST	$2.67 / $5.00
			Total spend: $7.67 Total loaded: $10.00

PINECONE

6	310	0.3%

10-minute throttling per alert type to prevent Discord spam during cascading failures.

What it shows:

Real-time: sessions, RAM, CPU, uptime

Sessions: queries, videos, errors, alerts

Users: total, returning, avg sessions/user

API Cost: tokens, spend, cycle budget

Pinecone: cached videos, vectors, fullness

Event log: filters, latency, cost per query

External links: LangSmith, Koyeb, GA

Auto-refresh every 30 seconds

Discord Alerting (6 scenarios):

`anthropic_api` - API errors, rate limits

`ip_blocked` - YouTube IP blocking

`proxy_down` - proxy connection failures

`pinecone_error` - embed/upsert/query errors

`uncaught_500` - unhandled exceptions

`budget_threshold` - 80% of $5 cycle

Event Log Detail

PINECONE

6	310	0.3%
CACHED VIDEOS	TOTAL VECTORS	INDEX FULLNESS

Recent Events

All | Queries | Videos | Errors | Sessions

Avg latency: 14.9s Tokens: 3.0k/686 Query cost: $0.02 Cache hit rate: 100%

Time (Europe/Berlin)	Type	IP	Tool	Latency ↑↓	Tokens	Cost	Detail
12 Mar 17:49:33	KEY	212.86.61.16	summarize_video	14.9s	3.0k/686	$0.02	"Can you provide a concise summary of the video co…
12 Mar 17:49:23	TOOL	–		135ms			summarize_video video=cdiD-9MMpb0 latency=135ms
12 Mar 17:49:08	VIDEO	212.86.61.16					cdiD-9MMpb0
12 Mar 17:49:05	VIDEO	212.86.61.16					e-gwvmhyU7A
12 Mar 17:49:02	VIDEO	212.86.61.16					dQw4w9WgXcQ
12 Mar 17:48:38	SESSION	–					active=1
12 Mar 17:36:44	VIDEO	212.86.61.16					cdiD-9MMpb0
12 Mar 17:36:39	VIDEO	212.86.61.16					e-gwvmhyU7A
12 Mar 17:36:23	VIDEO	212.86.61.16					dQw4w9WgXcQ
12 Mar 17:36:21	SESSION	–					active=1

EXTERNAL DASHBOARDS

LangSmith | Pinecone | Koyeb | Google Analytics | Discord

Auto-refresh: 30s | Last updated: 18:30:50

Event Log Features:

Filter tabs: All, Queries, Videos, Errors, Sessions

Summary bar: avg latency, tokens, cost, cache rate

Latency colour-coded: green <5s, yellow 5-15s, red >15s

Token display: formatted as 8.7k/0.5k (in/out)

Cost computed per query from token counts

Sortable latency column (click header)

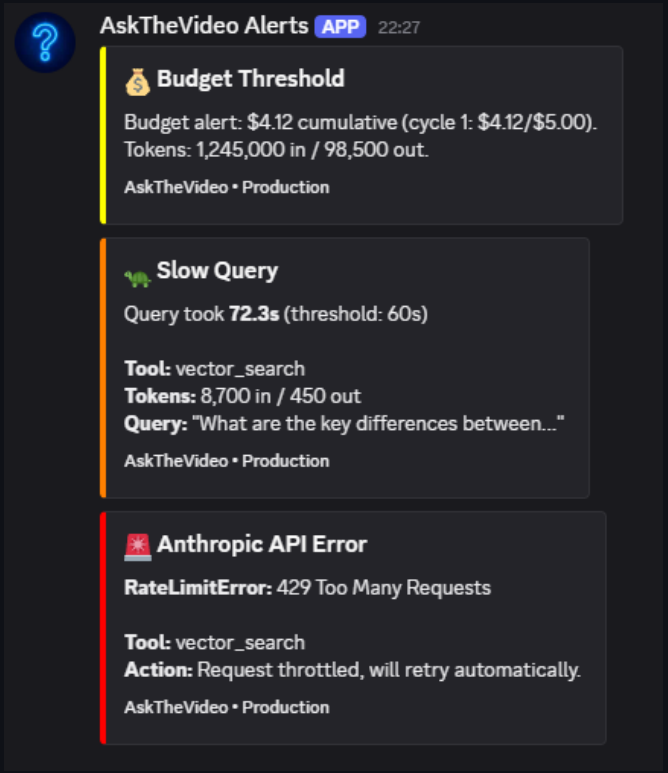
Timestamps in local timezone

Enriched event fields: tool, video_id, chunks, tier

Added today (Mar 12):

Anonymous user tracking (X-User-ID header)
Session persistence (sessionStorage)
Token offset env vars for cumulative display
Cycle-based budget system
Users card in admin dashboard
Enriched event log with all new columns

Discord Alerting

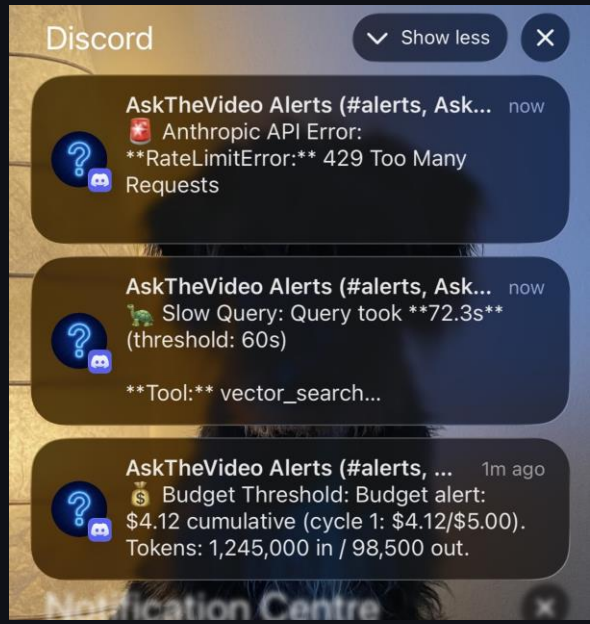


6 alert scenarios:

`anthropic_api`	API errors, rate limits, credit issues
`ip_blocked`	YouTube IP blocking detected
`proxy_down`	Proxy connection failures
`pinecone_error`	Embed/upsert/query errors
`uncaught_500`	Unhandled server exceptions
`budget_threshold`	80% of $5 cycle budget reached

Design decisions:

10-minute throttling per alert type
Prevents Discord spam during cascading failures
Direct calls at each error site (not a wrapper)
Specific alert_type for precise diagnostics



Mobile alerts

Real alerts from production: budget threshold ($4.12/$5.00), slow query (72.3s), and Anthropic rate limit (429). All received in real time on desktop and mobile.

Features: How They Work

SSE Streaming

asyncio.to_thread() + asyncio.Queue. Agent.stream() is synchronous, offloaded to thread pool. stream_mode='messages' for per-token output. Frontend reads data: lines, parses JSON, checks .token field.

Users see answer forming in real time instead of 20s spinner.

Voice Input

Browser Web Speech API with feature detection. Mic button shown only on Chrome/Edge/Safari. Firefox users see no button. Zero backend cost.

Hands-free interaction. Impressive in demos.

Voice Output

Browser SpeechSynthesis API. Speaker icon on each assistant message bubble. Tap to start, tap to stop.

Accessibility. Read answers while doing something else.

Chat Export

Client-side markdown file download. Pure frontend, no API needed. Formats conversation as user/assistant turns with timestamps.

Users can save research. Good for students taking notes.

Responsive Design

Tailwind CSS responsive breakpoints. Drawer sidebar on mobile. Touch-friendly UI elements. shadcn/ui components with built-in responsive behaviour.

Works on any device. Not just a desktop demo.

Features: How They Work (continued)

Session Persistence

Session ID stored in sessionStorage (key: atv_session_id). On reload, restored and sent as X-Session-ID header. Backend returns existing videos, history, limits. Stale sessions cleared automatically.

Page refresh no longer loses everything. Major UX improvement.

Conversation Memory

LangGraph MemorySaver stores conversation turns per thread_id. Follow-up questions reference earlier answers. Agent sees full context.

Natural multi-turn dialogue. 'Tell me more about that' works.

User Analytics

Anonymous UUID generated on first visit, stored in localStorage (atv_uid). Sent as X-User-ID on all requests. Backend upserts user record in Supabase: first_seen, last_seen, total_sessions, questions, videos.

Product analytics without cookies or consent banners.

Admin Dashboard

React /admin route with token auth (sessionStorage). Auto-refresh 30s. Reads GET /api/admin/metrics. Real-time stats, session counts, cost tracking, Pinecone stats, enriched event log with filters.

Monitor a production application. Not just build and forget.

All features implemented in ~10 days alongside the core AI pipeline, API layer, testing, and deployment.

Q&A Agent

LangGraph ReAct Agent

Model	Claude Sonnet 4.6
Framework	LangGraph + LangChain
Memory	MemorySaver (multi-turn conversation)
Streaming	SSE, token-by-token

The agent decides which tool to call based on the tool descriptions.

Vague descriptions: wrong routing

Precise descriptions: **100% routing accuracy**

across all initial test cases

The tool descriptions are the most important piece of code in the entire project.

Challenges

Streamlit --> React

Plan: Streamlit frontend (matches bootcamp tooling)

Problem:

Designed UI in Lovable (React). Tried to replicate in Streamlit.
Styling degraded. No SSE streaming. No voice APIs.
Poor mobile experience.

Decision:

Pivot to React + FastAPI.
All backend code survived untouched.
Memory: 90MB (Streamlit) to 35MB (FastAPI).

YouTube blocks cloud servers

Problem:

Every video load failed after deployment.
YouTube blocks transcript requests from datacenter IPs.

Tried and failed (half a day lost):

1. Cloudflare Worker proxy
2. Google Cloud Function proxy
3. yt-dlp fallback

All hit the same blocked endpoint.

Solution:

Webshare residential proxy. 10 lines of code.
Rotating residential IPs.